

EVENT-INVITATION

Codebase Cleanup, Performance & Security Audit

Repository: Ir-faan/Event-Invitation

Branch audited: main

Snapshot tree: 83d911e0b12490292eebe74fde7bef414f3bcfa1

Audit date: 16 September 2026

Prepared as a static source-code audit of the current GitHub repository.

It covers unused/dead material, runtime performance, storage/database efficiency, and security/privacy weaknesses.

Important scope note

This is a thorough repository-level audit, not a penetration test or production profiler. I inspected the current main-branch source, routes, database scripts, auth, media pipeline, dashboard, public invitation renderer and asset tree. Live Supabase data, Cloudflare analytics, production request traces and actual deployed environment variables were not accessible, so findings that depend on runtime state are explicitly marked conditional.

Executive summary

Top priority: stop leaking order contact data into public invitation clients

The stored InvitationConfig contains contact.name and contact.phone. The public [slug] page passes the full normalized config into PublishedInvitation, which is a client component. That means these customer-only fields can be serialized into the browser payload even though the UI does not visibly show them. Create a stripped PublicInvitationConfig before anything is sent to the client.

Top priority: protect public creation/upload endpoints from bots

The public invitation creation route has no application-level rate limit or bot challenge. A valid upload capability can also be replayed for the same slot many times during its 30-minute lifetime, creating orphan storage objects before finalization. Add Cloudflare Turnstile + rate limits + one-use upload slots.

The highest-return improvements are:

- Strip admin/customer-only fields from public invitation data before client serialization.
- Add bot/rate-limit controls to /api/invitations and /api/invitations/media, and make each upload slot one-use.
- Stop loading full config JSON for every dashboard row; use summary-only server pagination and fetch full config only when an order is opened.
- Move landing-only CSS and DOM-patch client components out of the root layout; fold their behavior directly into the landing component.
- Parallelize photo compression/upload with bounded concurrency and move uploads directly to storage using short-lived signed upload URLs.
- Deduplicate/cachify public invitation reads and remove force-dynamic/no-store where safe, with explicit invalidation on admin updates.
- Optimize the two ivory envelope images (roughly 1.9 MB and 2.0 MB) and lazy-load below-the-fold invitation imagery.

Audit scorecard

Area	Current state	Main concern	Recommended direction
Unused / debt	Moderate	Legacy landing code and patch-style DOM/CSS layering	Consolidate and delete verified legacy files
Dashboard performance	Needs improvement	Up to 250 full configs fetched for a summary table	Summary query + server pagination
Save/upload performance	Needs improvement	Compression and uploads are largely sequential; app proxies bytes	Bounded parallelism + direct signed uploads
Public invitation speed	Needs improvement	Dynamic/no-store + duplicate DB reads + full client renderer	Request dedupe, cache/invalidate, split public renderer

Area	Current state	Main concern	Recommended direction
Authentication	Reasonably strong	Remote user + admin allowlist checks on each protected request	Local JWT verification / cached admin authorization
Abuse resistance	Weak	No public rate limiting / Turnstile; upload capability replay	WAF/rate limits + one-use slot state
Privacy	High-priority issue	Contact fields can reach public browser; bucket is public	Public DTO + private/signed media strategy
Database integrity	Good foundation	Transactions exist, but storage and DB cleanup can diverge	Idempotent cleanup/retry job and staging lifecycle

Audit scope and methodology

Reviewed areas: application routing, root layout, landing page, templates, invitation designer, mobile/public renderer, dashboard, admin authentication, customer submission API, media API, Supabase access wrapper, database schema/migrations, storage design, image assets and deployment worker.

- Repository-wide recursive tree of main, including file sizes and public assets.
- Active route entry points and the components they import.
- Database table definitions, indexes, RLS setup, transactional media RPCs and obsolete-column migration.
- Customer save path: validation -> capability issue -> client compression -> media upload -> final database commit.
- Admin path: authentication proxy -> order list/detail -> update/deploy/duplicate/delete -> media cleanup.
- Public invitation path: slug lookup -> metadata -> normalized configuration -> client-side renderer.

Confidence labels

Confirmed = directly evidenced by imports/schema/code paths. High = no active reference found and architecture strongly indicates removal is safe, but run tests/build after deletion. Conditional = depends on whether a migration has already been applied or whether a production-only reference exists outside the repository.

1. Unused, redundant or unnecessary material

U01 Legacy landing component: components/landing-experience.tsx

Priority: P2 **Confidence:** High

Evidence: app/page.tsx renders LandingExperienceRefresh, while the older LandingExperience component remains as a separate ~28 KB source file.

Why it matters: It duplicates much of the landing markup, arrays, effects and wording. Keeping two generations of the home page increases maintenance burden and makes future searches/refactors misleading.

Recommended action: Delete components/landing-experience.tsx after running the build/tests and verifying no non-main branch or test fixture imports it.

Expected result: Less dead code and less chance of future edits landing in the wrong component.

U02 Unreferenced patch stylesheet: app/rose-hero-background-fix.css

Priority: P3 **Confidence:** High

Evidence: The Rose Afterglow route imports its module CSS plus seven route-specific patch stylesheets, while the root layout imports rose-scratch-sparkles.css but not rose-hero-background-fix.css.

Why it matters: A standalone fix stylesheet that is no longer imported is repository clutter and can mislead future work.

Recommended action: Delete it after a final text/reference search and visual check of /templates/rose-afterglow.

Expected result: Cleaner template styling and fewer stale fixes.

U03 Five palette-specific Bismillah images are redundant

Priority: P3 **Confidence:** Confirmed by current mapping

Evidence: bismillahAssets maps beige, olive, dusty-blue, burgundy, pink and lilac all to builder-bismillah-beige.webp. The other five builder-bismillah-*.webp files remain in public/images.

Why it matters: These files add repository/deploy payload without being selected by the current designer/public renderer.

Recommended action: Remove builder-bismillah-olive.webp, -dusty-blue.webp, -burgundy.webp, -pink.webp and -lilac.webp after build verification. Keep builder-bismillah-beige.webp.

Expected result: About 85 KB removed from source/deploy artifacts and one less family of assets to maintain.

U04 DOM patch components are unnecessary architecture, even though they are currently mounted

Priority: P2 **Confidence:** Confirmed

Evidence: Root layout mounts TemplateCardRouter, FooterSocialLinkFix and LandingMainPolish globally. These components mutate already-rendered landing DOM, replace links/children, rewrite text, clone buttons and add listeners.

Why it matters: The same page is effectively rendered twice: first by React, then partially rewritten by effects. They also ship on non-landing routes because they live in the root layout.

Recommended action: Move the final hrefs, wording, social anchors, marquee asset list and mobile CTA directly into LandingExperienceRefresh. Then delete these patch components and remove them from app/layout.tsx.

Expected result: Smaller client bundle, less hydration work, fewer brittle selector dependencies.

U05 Landing CSS patch stack should be consolidated and route-scoped

Priority: P2 **Confidence:** Confirmed unnecessary global scope

Evidence: app/layout.tsx imports many landing-only and template-fix CSS files globally, including landing-refresh, navbar cleanup, contact polish, gallery fix, marquee clean, mobile polish, palette polish and more.

Why it matters: Even when selectors do not match, every route pays the CSS transfer/parse cost and future specificity conflicts become harder to reason about.

Recommended action: Move landing-only imports into the home route (or a home-only layout) and merge stable patch files into one maintained landing stylesheet. Move template fixes into their template routes.

Expected result: Lower CSS bytes on login/dashboard/designer/public invitations and simpler cascade behavior.

U06 Obsolete database objects are explicitly identified by your migrations

Priority: P3 **Confidence:** Conditional on production migration state

Evidence: supabase/remove-obsolete-columns.sql and dashboard-migration.sql remove edit_token_hash, updated_at, invitations_edit_token_hash_idx, invitations_status_updated_at_idx, invitations_set_updated_at trigger and set_invitation_updated_at() function.

Why it matters: If these migrations have not been run in production, the objects are confirmed dead schema baggage.

Recommended action: Check information_schema / pg_indexes in production. If still present, run the supplied cleanup migration in a backup-tested maintenance window. Do not remove created_at, deployed_at or inactive_at; the repo explicitly still uses them.

Expected result: Smaller schema surface and fewer obsolete indexes/triggers.

U07 Abandoned pre-finalization uploads can become storage garbage

Priority: P1 **Confidence:** Confirmed design risk

Evidence: Customer images are uploaded to Storage before create_invitation_with_media commits metadata. Cleanup only happens if the browser calls the cancel path; closing the tab, losing connectivity, or bot abuse can leave objects with no invitation_media row.

Why it matters: This creates invisible storage cost and makes “unused assets” accumulate outside Git.

Recommended action: Use a staging prefix/bucket and scheduled lifecycle cleanup: delete objects older than e.g. 1-2 hours that have no invitation_media row. Make the cleanup idempotent and observable.

Expected result: Prevents orphan storage growth and makes failed submissions self-healing.

U08 Candidate legacy envelope assets should be verified before deletion

Priority: P3 **Confidence:** Medium

Evidence: The repository contains forest-envelope-desktop.webp and forest-envelope-mobile.webp. The active Coastal Reverie component uses ivory-envelope-*.webp, and the current builder uses builder-envelope-* assets. Repository code search returned no forest-envelope references, although GitHub reported the search index as incomplete.

Why it matters: Likely old assets, but the incomplete search result means deletion should not be automatic.

Recommended action: Run a local ripgrep after cloning: rg -n "forest-envelope" . If no reference is found, delete both and verify the template/gallery build. Apply the same process to other suspicious one-off assets before removal.

Expected result: Potentially removes another ~280 KB without risking active visuals.

2. Performance and scalability improvements

P01 Dashboard list downloads full config JSON for up to 250 orders

Priority: P1 **Confidence:** Confirmed

Evidence: listInvitationOrders selects id/status/slug/dates/price plus config, limit 250, then summarizeOrder() is applied. The dashboard only needs summary data until openOrder(id) performs a separate detail request.

Why it matters: As orders grow, list response size, JSON parsing, server memory and browser filtering all grow with the full invitation configuration, including sections and image URLs.

Recommended action: Create a dashboard summary view/RPC or PostgREST select that extracts only customer/couple/event summary fields plus status/price/dates. Paginate on the server (e.g. 25-50 rows) and send search/status/sort as query parameters. Keep GET?id for the full config.

Expected result: Much faster dashboard first load, lower Supabase egress and predictable scaling beyond 250 orders.

P02 InvitationDesigner is bundled into the initial dashboard

Priority: P2 **Confidence:** Confirmed

Evidence: components/invitation-dashboard.tsx imports InvitationDesigner at module load even though the initial screen is the orders table. InvitationDesigner itself imports the phone preview, upload pipeline and many controls.

Why it matters: Users pay for the large editor bundle even when they only review/filter/deploy from the table.

Recommended action: Dynamically import the editor when an order is opened, or move editing to /dashboard/orders/[id] with route-level code splitting.

Expected result: Smaller initial dashboard JavaScript and faster interaction readiness.

P03 Photo optimization is sequential and CPU-heavy on the browser main thread

Priority: P1 **Confidence:** Confirmed

Evidence: preparePendingPhotos iterates files sequentially. preparePhoto creates Image + canvas and may try up to four dimensions x three WebP quality values.

Why it matters: Multiple wedding photos can block mobile devices for a long time, consume large decoded-image memory and make Save appear slow.

Recommended action: Use createImageBitmap and OffscreenCanvas/Web Worker where available. Estimate a target dimension once, then do 1-2 encodes rather than up to 12. Process at bounded concurrency (2-3 images) and cap decoded pixel count, not just compressed file bytes.

Expected result: Faster saves, less UI jank and fewer mobile crashes.

P04 Uploads are fully sequential

Priority: P1 **Confidence:** Confirmed

Evidence: uploadPendingPhotos loops every slot/file and awaits each fetch before starting the next. A Glimpse section can contain multiple images and the API permits up to 32.

Why it matters: Network latency is multiplied by photo count.

Recommended action: Upload with bounded concurrency (3-4 simultaneous files) using Promise workers. Preserve slot ordering in the returned mapping and expose progress per file.

Expected result: Large multi-photo invitations save substantially faster without flooding the network.

P05 Every upload is proxied through the application edge worker

Priority: P1 **Confidence:** Confirmed

Evidence: /api/invitations/media parses multipart, materializes file.arrayBuffer(), then sends the bytes to Supabase Storage. Admin upload does the same.

Why it matters: Each image traverses client -> app worker -> Supabase, doubling bandwidth and adding worker memory/CPU.

Recommended action: Have the server issue a short-lived signed upload URL/token for exactly one path/slot. Upload browser -> Supabase directly, then finalize with server-verified receipts/metadata. Keep strict path/slot constraints.

Expected result: Lower edge cost, lower latency, fewer 413/timeouts and better scalability.

P06 Per-file customer upload performs an extra invitation lookup

Priority: P2 **Confidence:** Confirmed

Evidence: The customer media route calls `getInvitationOrder(invitationId)` for every uploaded photo to confirm the order has not already been committed.

Why it matters: For N photos this adds N database round trips on top of storage uploads.

Recommended action: When switching to one-use signed direct uploads, enforce the staging capability state once. If keeping the proxy route, cache the “not committed” state briefly per invitation ID or batch uploads.

Expected result: Fewer Supabase requests during save.

P07 Public invitation is force-dynamic and performs duplicate slug reads

Priority: P1 **Confidence:** Confirmed

Evidence: `app/[slug]/page.tsx` exports `dynamic="force-dynamic"`. `generateMetadata()` calls `getPublicInvitationBySlug()`, and the page function calls it again; the helper uses `cache:"no-store"`.

Why it matters: Every guest view can trigger two Supabase reads and cannot benefit from application caching. Shared wedding links can receive bursts of traffic.

Recommended action: Wrap the loader in `React cache()` to dedupe within a request. Then use a short cache/tag for active invitation data and invalidate/revalidate on admin save/deploy/deactivate. Remove `force-dynamic` if the deployment adapter supports the chosen cache strategy.

Expected result: Potentially halves DB reads immediately and makes popular invitations much cheaper/faster.

P08 Public invitation imports the full designer stylesheet

Priority: P2 **Confidence:** Confirmed

Evidence: `app/[slug]/page.tsx` imports `app/design-invitation/design-invitation.css` plus `published-invitation.css`. The designer stylesheet is ~88 KB in source.

Why it matters: Guests need only the published invitation styles, not form/editor layout rules.

Recommended action: Extract shared invitation-renderer CSS into a dedicated file used by preview + public view; keep builder/editor styles separate.

Expected result: Less CSS transfer/parse work on every guest visit.

P09 PublishedInvitation shares the large client preview module

Priority: P2 **Confidence:** Confirmed

Evidence: `PublishedInvitation` and `InvitationPhonePreview` live in the same “use client” module with scratch canvas, resize observers, focus scrolling, preview shell and many section utilities.

Why it matters: The public route can ship preview/editor-oriented client code that guests do not need.

Recommended action: Create a separate public invitation renderer module. Keep static sections server-renderable where possible; lazy-load only interactive opening/scratch behavior when configured.

Expected result: Smaller public JS bundle and quicker hydration on older phones.

P10 Expiration is implemented as a write during reads

Priority: P2 **Confidence:** Confirmed

Evidence: listInvitationOrders calls expirePastInvitations() before every list. getPublicInvitationBySlug also patches an expired record when a visitor requests it.

Why it matters: Read traffic generates write traffic and adds latency/failure points.

Recommended action: Filter public lookup by status=active and active_until >= today without mutating. Move status cleanup to a scheduled job/cron, or update on dashboard load at a much lower frequency.

Expected result: Lower write load and simpler public request path.

P11 Slug generation can issue many sequential uniqueness queries

Priority: P3 **Confidence:** Confirmed

Evidence: createUniqueInvitationSlug tests candidates in a loop up to 100 times; duplicate slug creation has a similar loop.

Why it matters: Usually cheap, but collisions or bulk duplication can cause many round trips.

Recommended action: Generate a readable base plus short random suffix, insert under the existing unique index, and retry only on unique-constraint conflict.

Expected result: One database round trip in the common case.

P12 Duplicating an order copies each media file through the edge app sequentially

Priority: P1 **Confidence:** Confirmed

Evidence: Dashboard duplicate downloads every source object via public URL into arrayBuffer and re-uploads it, one file at a time.

Why it matters: A 20-photo order causes 40 storage network operations plus edge byte handling and can hit execution limits.

Recommended action: Use Supabase server-side object copy if available, with bounded concurrency. If physical copies are not required, consider immutable shared media with reference counting; otherwise keep independent paths but copy within storage.

Expected result: Much faster Duplicate action and lower worker bandwidth/memory.

P13 Two active envelope assets are extremely large

Priority: P1 **Confidence:** Confirmed

Evidence: ivory-envelope-desktop.webp is ~1.88 MB and ivory-envelope-mobile.webp is ~2.01 MB. Coastal Reverie loads them in the opening sequence.

Why it matters: The opening can delay first meaningful content, especially on mobile; the mobile file is paradoxically larger than desktop.

Recommended action: Re-export at the actual rendered resolution, target roughly 150-350 KB each if visual quality permits, and consider AVIF/WebP variants. Preload only the correct responsive source.

Expected result: A multi-megabyte reduction on the template opening path.

P14 Raw img usage misses the existing Cloudflare/vinext optimizer

Priority: P2 **Confidence:** Confirmed

Evidence: worker/index.ts exposes /_vinext/image image optimization, but active components predominantly render plain elements.

Why it matters: Large hero/gallery images are served at source dimensions instead of device-appropriate widths/formats.

Recommended action: Use the framework Image component where supported by vnext, or call the optimization path for large photos with width/srcset/sizes. At minimum add loading="lazy" and decoding="async" below the fold.

Expected result: Lower image bytes and improved LCP on phones.

P15 Global landing effects and CSS load on every route

Priority: P2 **Confidence:** Confirmed

Evidence: Root layout mounts four landing-oriented client components and imports many landing/template global styles before rendering any child route.

Why it matters: Dashboard/login/designer/public invite visitors pay for unrelated page code and style parsing.

Recommended action: Route-scope them to the home page. Root layout should contain only true site-wide metadata/base styles/providers.

Expected result: Cross-site improvement to bundle size and hydration work.

P16 Admin authentication performs multiple remote checks per protected request

Priority: P2 **Confidence:** Confirmed

Evidence: getAdminSession verifies the access token by calling Supabase /auth/v1/user and then queries dashboard_admins; proxy executes this for protected pages/API calls.

Why it matters: Opening the dashboard and subsequent API requests repeatedly incur auth/network round trips.

Recommended action: Verify Supabase JWT locally using JWKS and expiry/issuer/audience checks. Put admin authorization in a signed custom claim or cache the allowlist lookup for a short TTL with explicit revocation strategy.

Expected result: Faster protected requests without weakening authentication.

P17 Countdown/effects can trigger broad client rerenders and continuous animation work

Priority: P3 **Confidence:** High

Evidence: Template components use one-second state updates and multiple decorative animated spans/observers. Public preview also performs name measurement via layout probes and ResizeObserver.

Why it matters: Older phones and power-saving devices can spend CPU on decoration instead of content.

Recommended action: Isolate countdown into a tiny memoized component, pause animations when document.hidden, honor prefers-reduced-motion, and prefer CSS sizing (clamp/container queries) before JS text measurement.

Expected result: Smoother invitations and better battery usage.

3. Security, abuse and privacy weaknesses

S01 Customer contact details can be exposed in public browser payload

Priority: P0 **Confidence:** Confirmed architectural exposure

Evidence: InvitationConfig contains contact.name and contact.phone. The public slug page passes normalizeInvitationConfig(order.config) to PublishedInvitation, and that renderer is a client component. React/Next must serialize client-component props to the browser.

Why it matters: Guests with the invitation link can potentially inspect customer/order contact data that is not intended to be visible. This is unnecessary PII exposure.

Recommended action: Define PublicInvitationConfig containing only guest-visible fields. Strip contact and any future admin-only data on the server before passing props. Add an automated test that public HTML/RSC payload never contains a known test phone number/customer name.

Expected result: Removes a direct privacy leak and reduces public payload size.

S02 Public invitation creation has no application-level bot protection

Priority: P1 **Confidence:** Confirmed

Evidence: POST /api/invitations is unauthenticated by design and validates content, but no Turnstile/CAPTCHA, IP rate limit, device/session quota or abuse token is enforced in the application.

Why it matters: Bots can create large numbers of pending orders, trigger slug checks, consume database rows and generate upload capabilities.

Recommended action: Add Cloudflare Turnstile to Save Design, verify token server-side, and rate-limit by IP + short-lived browser/session key. Add daily per-IP/order quotas and WAF rules for abnormal traffic.

Expected result: Protects database/storage budget and dashboard from spam orders.

S03 Upload capability can be replayed repeatedly for the same slot

Priority: P0 **Confidence:** Confirmed

Evidence: A signed capability contains id/folder/slots/expires. The upload route only checks that the requested slot is in capability.slots. It does not record that a slot has already been used. Each request generates a new random storage path.

Why it matters: During the 30-minute validity window an attacker can repeatedly upload ≤ 5 MB objects using one allowed slot, creating unbounded orphan storage without finalizing an invitation.

Recommended action: Give every capability a jti and persist per-slot state in Durable Object/KV/DB, or issue one short-lived signed upload URL per exact object path and mark it consumed. Also rate-limit by capability/id/IP and delete/reject replacements deterministically.

Expected result: Closes the clearest storage-amplification vector.

S04 Server upload limit is much larger than the normal client target

Priority: P1 **Confidence:** Confirmed

Evidence: Client compression targets ≤ 900 KB, but the server accepts files up to 5 MB.

Why it matters: Attackers bypassing the UI can consume over 5x the expected bandwidth/storage per object; combined with replay this is material.

Recommended action: After direct-upload redesign, enforce a server/storage limit close to the product requirement (e.g. 1-1.5 MB) and reject excessive dimensions. Keep 5 MB only for a separately authenticated raw-upload workflow if truly necessary.

Expected result: Reduces abuse amplification and normalizes resource usage.

S05 File type is trusted from multipart MIME header rather than validated content

Priority: P1 **Confidence:** Confirmed

Evidence: Both media routes map file.type to an extension and storage Content-Type but do not sniff magic bytes or decode the uploaded file server-side.

Why it matters: Arbitrary bytes can be labeled image/webp/jpeg/png. A public storage bucket then serves attacker-controlled content under an image MIME type.

Recommended action: Validate magic bytes and preferably decode/re-encode images in a trusted image pipeline. If direct upload is used, quarantine/stage then verify before making the object guest-accessible.

Expected result: Stronger content integrity and less risk from malformed/polyglot uploads.

S06 Compressed-byte limit does not protect against huge decoded image dimensions

Priority: P1 **Confidence:** Confirmed

Evidence: Client validation caps original file bytes at 5 MB but does not inspect pixel count before browser decode/canvas work.

Why it matters: A highly compressed enormous image can cause memory pressure or browser crashes during preview/optimization.

Recommended action: Read dimensions before full processing where possible and enforce a pixel ceiling (for example 20-30 megapixels), then downscale via createImageBitmap/worker.

Expected result: Prevents image decompression/resource exhaustion on client devices.

S07 Public storage bucket exposes uploaded wedding media by URL

Priority: P1 **Confidence:** Confirmed design choice

Evidence: supabase/setup.sql creates invitation-media as public=true; config contains public_url values used by invitations.

Why it matters: Anyone who obtains an object URL can retrieve the photo independently of invitation status. Links can survive after an invitation is deactivated if objects remain.

Recommended action: For stronger privacy, use a private bucket and issue signed URLs/proxy responses only for active invitations. If public storage is retained for performance, use unguessable paths, cleanup on deactivation policy as appropriate, and clearly document the privacy model.

Expected result: Aligns media visibility with the invitation lifecycle.

S08 Media signing is coupled to the Supabase service-role key

Priority: P2 **Confidence:** Confirmed

Evidence: media-submission.ts derives the HMAC key from SUPABASE_SERVICE_ROLE_KEY.

Why it matters: The database superuser credential now doubles as a token-signing root. Rotating it invalidates in-flight receipts/capabilities, and compromise has a wider blast radius.

Recommended action: Introduce a dedicated MEDIA_SIGNING_SECRET (versioned for rotation) stored only server-side. Keep service-role key exclusively for Supabase privileged API calls.

Expected result: Cleaner key separation and safer rotation.

S09 Admin login relies mostly on upstream throttling

Priority: P2 **Confidence:** Confirmed

Evidence: The login route enforces same-origin and size validation and handles Supabase 429, but does not add app/edge rate limiting per IP/email.

Why it matters: A public /login endpoint can still be brute-force or credential-stuffing traffic against Supabase Auth, consuming auth quota.

Recommended action: Add Cloudflare rate limiting for POST /api/admin-session, escalating delays/temporary blocks and alerting. Keep generic credential failure text.

Expected result: Reduces password-guessing and auth service abuse.

S10 Security headers are not configured in application code

Priority: P2 **Confidence:** High

Evidence: next.config.ts is empty and worker/index.ts forwards normal requests to the app handler without adding CSP, Permissions-Policy or other hardening headers.

Why it matters: The app relies on platform defaults and browser baseline behavior. A future injection bug would have fewer containment layers.

Recommended action: Add deployment-compatible headers: Content-Security-Policy tailored to Supabase, maps and social links; X-Content-Type-Options: nosniff; Referrer-Policy; Permissions-Policy; frame-ancestors via CSP. Test template embeds before enforcing.

Expected result: Defense in depth against XSS/content sniffing/clickjacking and referrer leakage.

S11 Storage deletion is best-effort and can diverge from database state

Priority: P2 **Confidence:** Confirmed

Evidence: Order DELETE removes the database row first, then attempts object deletion in a try/catch. If Storage deletion fails, the order is gone but files remain. Similarly staged failures can orphan objects.

Why it matters: Private photos can persist without metadata/visibility in the dashboard and cost storage.

Recommended action: Use an idempotent deletion job/queue: mark order pending deletion, enqueue paths, retry object deletion until confirmed, then remove/finalize metadata. Also run periodic orphan reconciliation.

Expected result: Reliable data lifecycle and easier incident cleanup.

S12 Public request body is validated after JSON parsing

Priority: P2 **Confidence:** Confirmed

Evidence: /api/invitations checks JSON.stringify(config).length <= 750,000 only after request.json() has parsed the body. Unlike login, it does not pre-check Content-Length.

Why it matters: Oversized bodies can consume edge memory/CPU before your application rejects them.

Recommended action: Reject unreasonable Content-Length before request.json(); also configure Cloudflare/request body limits and cap nested config sizes in validation.

Expected result: Reduces low-cost request-body DoS pressure.

Security and integrity measures already done well

- Admin dashboard routes are guarded by proxy.ts and non-GET mutations require same-origin requests.
- Admin access is not granted merely by a Supabase Auth login; dashboard_admins is a server-only allowlist.
- Auth cookies are HttpOnly and SameSite=Strict, with Secure enabled under HTTPS.
- Customer configuration has explicit shape/length/count validation and Mauritian phone validation.
- Customer media finalization uses signed receipts and an HMAC capability rather than trusting client-supplied storage paths.

- Database RPCs create/update invitation + invitation_media rows transactionally and are revoked from anon/authenticated roles.
- Slug has a unique database index; media paths are unique; RLS is enabled on primary tables.
- The client canvas recompression strips most EXIF metadata from images that are actually recompressed, which is privacy-positive.

4. Recommended implementation roadmap

Phase	Timing	Work
Phase 1 - security blockers	Before wider public traffic	S01 public DTO/PII stripping; S02 Turnstile/rate limiting; S03 one-use upload slots; S04 tighter upload size; S05 file-content validation.
Phase 2 - fastest performance wins	Next implementation batch	P01 summary-only dashboard query/server pagination; P02 lazy-load editor; P07 dedupe public DB read; P13 re-export ivory envelopes; P15 route-scope landing CSS/effects.
Phase 3 - upload architecture	After Phase 1	P03 worker-based compression; P04 bounded parallel uploads; P05 direct signed uploads; lifecycle cleanup for abandoned staging objects.
Phase 4 - public invitation renderer	After upload work	P08 split CSS; P09 dedicated public renderer; cache + invalidation; image optimization/lazy loading; reduced-motion support.
Phase 5 - cleanup and maintainability	After tests are green	Delete U01/U02/U03, fold U04 patches into source markup, consolidate U05 CSS, verify candidate legacy assets with local rg, confirm obsolete DB migration state.

Suggested target architecture for saving an invitation

1. Browser validates files and dimensions; image compression runs off the main thread with bounded concurrency.
2. Browser submits configuration + desired slots + Turnstile token to the server.
3. Server validates the config, rate limits, generates the order ID/slug, and issues exactly one short-lived signed upload target per slot.
4. Browser uploads compressed files directly to a staging/private storage path in parallel (bounded to 3-4).
5. Server finalization verifies every expected object (path, size, MIME/content, slot) and performs the transactional invitation + media metadata commit.
6. A lifecycle job removes expired staging objects that were never finalized.
7. Admin updates use the same media primitive; published invitations receive a stripped guest-visible config only.

Suggested target architecture for the dashboard

- GET /api/dashboard/orders?page=1&pageSize=25&status=pending&search=... returns only summaries and total/count metadata.
- GET /api/dashboard/orders?id=<uuid> returns the full config only when editing/previewing a specific order.
- InvitationDesigner is dynamically imported only after a detail is opened.
- Status counts come from a compact aggregate query rather than scanning all downloaded rows in the browser.
- Expired-status maintenance is a scheduled job rather than a side effect of every list/public read.

5. Cleanup checklist

Action	Path/object	Reason / guardrail
Safe to investigate/delete now	components/landing-experience.tsx	Legacy component; home uses landing-experience-refresh.tsx.
Safe to investigate/delete now	app/rose-hero-background-fix.css	No active import found in root or Rose route.
Safe to investigate/delete now	public/images/builder-bismillah-{olive,dusty-blue,burgundy,pink,lilac}.webp	All palettes currently map to builder-bismillah-beige.webp.
Delete after integrating behavior	components/template-card-router.tsx	Render correct target href directly.
Delete after integrating behavior	components/footer-social-link-fix.tsx	Render semantic <a> social links directly.
Delete after integrating behavior	components/landing-main-polish.tsx	Move final marquee/wording/social/mobile behavior into the landing component.
Consolidate	app/landing-*.css patch files	Keep final rules but merge and route-scope.
Conditional DB cleanup	edit_token_hash, updated_at and old indexes/trigger/function	Already listed in remove-obsolete-columns.sql; only if still present in live DB.
Verify locally first	public/images/forest-envelope-desktop.webp + forest-envelope-mobile.webp	No reference found by GitHub code search, but search index reported incomplete.
Automated cleanup	Uncommitted media objects older than upload capability lifetime	Not Git files; storage objects without invitation_media row.

6. Regression tests to add before/while optimizing

- Public privacy test: seed config with contact.phone = 51234567 and assert the public HTML/RSC response does not contain it.
- Upload capability replay test: second upload to the same one-use slot must fail or replace deterministically without leaving an orphan.

- Bot protection test: invalid/missing Turnstile token is rejected; rate-limit threshold returns 429.
- Upload content test: a text/HTML payload labeled image/webp is rejected.
- Dimension test: oversized-pixel image is rejected before expensive processing.
- Dashboard API test: list endpoint does not contain config; detail endpoint does.
- Pagination test: create >250 orders and prove every order is reachable through server pagination/search.
- Public cache test: update/deactivate an invitation and verify cache invalidation immediately changes guest output.
- Media lifecycle test: failed submission staging objects are removed by cleanup job; committed media is never removed.
- Delete lifecycle test: simulated Storage failure is retried and leaves an observable cleanup record, not a silent orphan.
- Asset/reference test in CI: detect unreferenced public assets and unexpected root-level CSS imports.
- Bundle budget: fail CI if public invitation/landing/dashboard JS or CSS exceeds agreed gzip thresholds.

7. Measurements to collect after changes

Flow	Measure
Landing	LCP, INP, CLS, JS transferred, CSS transferred, image bytes, number of image requests
Designer save	median/p95 compression time, upload time per file, total save time, 413/5xx rate
Dashboard	TTFB, list payload bytes, Supabase query duration, order count, p95 load time
Public invitation	TTFB, DB reads/view, cache hit rate, JS/CSS/image bytes, LCP on mobile
Abuse	orders/IP/day, capabilities issued, uploads/capability, rejected repeats, Turnstile failures, 429 count
Storage	total bytes, orphan staging bytes, cleanup success/retry count, media objects without DB metadata

8. Source map used for this audit

Key files inspected directly on main:

- app/layout.tsx
- app/page.tsx
- app/[slug]/page.tsx
- app/design-invitation/page.tsx
- app/dashboard/page.tsx
- app/api/invitations/route.ts
- app/api/invitations/media/route.ts
- app/api/admin-session/route.ts
- app/api/dashboard/orders/route.ts

- app/api/dashboard/orders/media/route.ts
- proxy.ts
- worker/index.ts
- components/landing-experience.tsx
- components/landing-experience-refresh.tsx
- components/landing-main-polish.tsx
- components/template-card-router.tsx
- components/footer-social-link-fix.tsx
- components/invitation-designer.tsx
- components/invitation-dashboard.tsx
- components/invitation-phone-preview.tsx
- components/coastal-reverie-invitation.tsx
- components/rose-afterglow-invitation.tsx
- lib/invitation-designer.ts
- lib/invitation-orders-server.ts
- lib/invitation-validation.ts
- lib/photo-upload.ts
- lib/media-submission.ts
- lib/admin-session.ts
- lib/supabase-server.ts
- supabase/setup.sql
- supabase/dashboard-migration.sql
- supabase/media-commit-migration.sql
- supabase/remove-obsolete-columns.sql
- supabase/admin-auth.sql
- package.json
- next.config.ts

Repository reference: github.com/Ir-faan/Event-Invitation

Audit snapshot note: The repository tree was read recursively at main tree SHA 83d911e0b12490292eebe74fde7bef414f3bcfa1. If main changes after this audit, rerun the unused/reference and security checks before deleting files or applying schema changes.

9. Priority matrix - what I would fix first

#	ID	Change	Category	Benefit	Effort
1	S01	Strip contact/admin fields from public config	Security/privacy	Very high	Small-medium
2	S03	One-use upload slots + replay/rate protection	Security/cost	Very high	Medium
3	S02	Turnstile + public API rate limits	Security/cost	Very high	Medium

#	ID	Change	Category	Benefit	Effort
4	P01	Summary-only paginated dashboard API	Performance	High	Medium
5	P05/P04	Direct storage upload + bounded concurrency	Performance/cost	High	Medium-high
6	P07	Dedupe/cache public invitation query	Performance/cost	High	Small-medium
7	P13	Compress ivory envelope assets	Page speed	High	Small
8	P15/U04	Remove global landing patch machinery	Performance/maintainability	Medium-high	Medium
9	S07/S11	Media privacy/lifecycle cleanup	Privacy/operations	High	Medium-high
10	U01-U06	Delete/consolidate verified legacy code/assets/schema	Maintainability	Medium	Small-medium

Bottom line

The project already has a solid foundation: server-only service role usage, RLS, admin allowlisting, transactional invitation/media metadata commits and strong input validation. The biggest risks are not “bad code everywhere”; they are concentrated in public data exposure, abuse controls and a handful of expensive data/media paths. Fixing the first 6-8 items above will materially improve privacy, scalability and responsiveness without redesigning the product.